

Project: AI-Powered Cybersecurity Incident Reconstruction and Analysis System

What are we building?

We are building a system that takes cybersecurity logs from different sources and reconstructs what happened during a possible cyber incident.

Normally, a security analyst may have thousands of separate log entries:

- A file was downloaded
- A process was started
- A system file was modified
- A network connection was made
- Logs were deleted

The problem is that these events are fragmented and difficult to connect manually.

Our system will connect these events, understand their sequence and relationships, and generate an understandable incident timeline with explanations.

In simple words:

Raw cybersecurity logs → Connected events → Reconstructed cyber incident → Evidence-based explanation

Example

A user uploads:

- System logs
- Process logs
- Network logs

The system may find:

```
1. invoice.zip was downloaded
  ↓
2. The file was extracted
  ↓
```

3. setup.exe was executed
↓
4. PowerShell was started
↓
5. A Registry Run key was modified
↓
6. An external server was contacted
↓
7. Security logs were deleted

The system then presents this as a possible incident:

A suspicious file was downloaded and executed. The system was modified to maintain access, an external connection was established, and logs were later deleted, possibly to hide the activity.

The user can click on an event and see the evidence and reasoning behind the conclusion.

How the AI part works

We are mainly using **Symbolic AI**, rather than depending on a neural network or an LLM.

The system will:

1. Convert raw logs into structured events.
2. Connect related events using a graph.
3. Apply cybersecurity reasoning rules.
4. Generate conclusions from the available evidence.

For example:

```
Executable downloaded
      +
Executable launched
      +
Registry modified
      ↓
Possible persistence activity
```

The system can then explain:

"This conclusion was made because the executable was launched and a Registry Run key was modified shortly afterward."

The reasoning is therefore transparent and evidence-based.

Tech Stack

Backend / Web API

Python + FastAPI

- Receives uploaded log files.
- Processes requests.
- Connects the frontend with the analysis engine.

Data Processing

Pandas + Pydantic

- Pandas helps read and process CSV or structured log data.
- Pydantic helps define a standard format for every event.

Example:

```
Time: 09:10
Source: System Log
Action: File Download
Process: Chrome
Object: invoice.zip
```

AI / Reasoning

Python-based Rule Engine + Forward Chaining

- Stores cybersecurity rules.
- Starts with known events and derives new conclusions from them.

Example:

```
Known Events
  ↓
Apply Rules
  ↓
New Conclusions
  ↓
Apply More Rules
```

Graph-Based Reasoning

NetworkX

- Represents events as nodes.
- Represents relationships between events as connections.

Example:

Download → Extract → Execute → System Modification

This helps us identify related chains of activity.

Cybersecurity Knowledge

MITRE ATT&CK

- We can use documented attacker techniques as a knowledge source.
- A selected subset of these techniques can be represented as rules in our system.

Frontend / Web Application

React + TypeScript

- Provides the user interface.
- Allows users to upload logs and explore the reconstructed incident.

Visualization

React Flow / D3.js

- Displays the event graph and attack timeline visually.
- Allows users to click on events and inspect their evidence.

Storage

SQLite initially, PostgreSQL if required

- Stores uploaded data, extracted events, rules, and analysis results.

Complete System Flow

```
graph TD
    A[User uploads log files] --> B[Backend receives the files]
    B --> C[Logs are parsed and cleaned]
    C --> D[Important events are extracted]
    D --> E[Events are converted into a common format]
    E --> F[Related events are connected into a graph]
    F --> G[Reasoning rules are applied]
    G --> H[Possible incident activities are identified]
```

A timeline is reconstructed

↓

Evidence and reasoning are attached

↓

The result is shown in an interactive dashboard

Final Example of the Output

Input:

```
09:10 - File downloaded
09:11 - File extracted
09:12 - Executable launched
09:13 - Registry modified
09:14 - External connection made
09:15 - Logs deleted
```

Output:

Possible Cybersecurity Incident

- **Initial Access:** Suspicious file downloaded.
- **Execution:** The extracted executable was launched.
- **Persistence:** A Registry Run key was modified.
- **Communication:** An external network connection was established.
- **Defense Evasion:** Security logs were deleted.

Evidence:

The system shows which original log entries led to each conclusion.

The main idea

We are **not simply building another log viewer or malware detector**.

We are building a system that tries to answer:

"What happened during this cyber incident, how are the events connected, and what evidence supports that conclusion?"

The main innovation is turning fragmented cybersecurity logs into an understandable, explainable incident story.